

ITF23019: Parallel and Distributed Programming

Report for Lab 6: Thread Synchronization 1

Name: Emil Berglund

GitHub: EmilB04

Exercise 1:

- Subtask 1: Run the code several times and include the output
 - *Attached 3 screenshots:*

```

● Parallel-distributed-programming/lab6 on 🐚 main > /usr/bin/
re/21.0.9-macosx-aarch64/bin/java -XX:+ShowCodeDetailsInExce
buted-programming/lab6/lab6_code/target/classes emilbe.Main
Thread-0: 2
Thread-1: 10
Thread-1: 20
Thread-0: 4
Thread-1: 30
Thread-0: 6
Thread-1: 40
Thread-0: 8
Thread-1: 50
Thread-0: 10
Thread-1: 60
Thread-0: 12
Thread-1: 70
Thread-0: 14
Thread-1: 80
Thread-0: 16
Thread-1: 90
Thread-0: 18
Thread-1: 100
Thread-0: 20
🐚
○ Parallel-distributed-programming/lab6 on 🐚 main > |

```

```

● Parallel-distributed-programming/lab6 on 🐚 main >
rogramming/lab6 ; /usr/bin/env /Users/emilb/.vscode
-aarch64/bin/java -XX:+ShowCodeDetailsInExceptionMe
ibuted-programming/lab6/lab6_code/target/classes em
Thread-1: 10
Thread-0: 2
Thread-1: 20
Thread-0: 4
Thread-1: 30
Thread-0: 6
Thread-0: 8
Thread-1: 40
Thread-1: 50
Thread-0: 10
Thread-1: 60
Thread-0: 12
Thread-0: 14
Thread-1: 70
Thread-1: 80
Thread-0: 16
Thread-0: 18
Thread-1: 90
Thread-0: 20
Thread-1: 100
🐚
○ Parallel-distributed-programming/lab6 on 🐚 main > |

```

```

● Parallel-distributed-programming/lab6 on 🐚 main > /
.0-darwin-arm64/jre/21.0.9-macosx-aarch64/bin/java -
ments/GitHub/V2026/Parallel-distributed-programming/
d
Thread-0: 2
Thread-1: 10
Thread-1: 20
Thread-0: 4
Thread-0: 6
Thread-1: 30
Thread-0: 8
Thread-1: 40
Thread-0: 10
Thread-1: 50
Thread-0: 12
Thread-1: 60
Thread-0: 14
Thread-1: 70
Thread-0: 16
Thread-1: 80
Thread-0: 18
Thread-1: 90
Thread-0: 20
Thread-1: 100
🐚
○ Parallel-distributed-programming/lab6 on 🐚 main > |

```

- Subtask 2: Do you get the same output from all runs?
 - No, I do not get the same output each time. The key difference is the execution order varies between runs. This demonstrates non-deterministic behaviour due to threads not being synchronized.
- Subtask 3: Add synchronized keyword to synchronize two threads.
 - After adding the keyword *synchronized*, I always got the same output for all runs. On the other hand, when the code is synchronized it is approximately twice as slow as unsynchronized.

```

● Parallel-distributed-programming/lab6 on 🐚 main >
  allel-distributed-programming/lab6 ; /usr/bin/env
-1.52.0-darwin-arm64/jre/21.0.9-macosx-aarch64/bin
es -cp /Users/emilb/Documents/GitHub/V2026/Paralle
rget/classes emilbe.MonitorSynchronizeMethod
Thread-0: 2
Thread-0: 4
Thread-0: 6
Thread-0: 8
Thread-0: 10
Thread-0: 12
Thread-0: 14
Thread-0: 16
Thread-0: 18
Thread-0: 20
Thread-1: 10
Thread-1: 20
Thread-1: 30
Thread-1: 40
Thread-1: 50
Thread-1: 60
Thread-1: 70
Thread-1: 80
Thread-1: 90
Thread-1: 100
○ Parallel-distributed-programming/lab6 on 🐚 main >

```

Exercise 2: producerConsumer

- Subtask 1: Run package *producerConsumer* and include output
 - Due to the length of the output, I will not include the whole console. Below are two screenshots, one showing the beginning, and one showing the end.

```
Parallel-distributed-programming/Lab6 on J main > cd /Users/emilb/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin
+ShowCodeDetailsInExceptionMessages -cp /Users/emilb/Library/Java/JavaVirtualMachines/temurin-21.jdk/Contents/Home/bin
Support/Code/User/workspaceStorage/2335ad49a886886ea66d6at.java/jdt_ws/Lab6_c8ffa991/bin producerConsumer.Main
Put: 1
Put: 2
Put: 3
Put: 4
Put: 5
Put: 6
Put: 7
Put: 8
Put: 9
Put: 10
Put: 11
Put: 12
Put: 13
Put: 14
Put: 15
Put: 16
Put: 17
Put: 18
Put: 19
Put: 20
Put: 21
Put: 22
Put: 23
Put: 24
Put: 25
Put: 26
Put: 27
Put: 28
Put: 29
Put: 30
```

```
Remove: 32:
Remove: 31:
Remove: 30:
Remove: 29:
Remove: 28:
Remove: 27:
Remove: 26:
Remove: 25:
Remove: 24:
Remove: 23:
Remove: 22:
Remove: 21:
Remove: 20:
Remove: 19:
Remove: 18:
Remove: 17:
Remove: 16:
Remove: 15:
Remove: 14:
Remove: 13:
Remove: 12:
Remove: 11:
Remove: 10:
Remove: 9:
Remove: 8:
Remove: 7:
Remove: 6:
Remove: 5:
Remove: 4:
Remove: 3:
Remove: 2:
Remove: 1:
Remove: 0:
Parallel-distributed-programming/Lab6 on J main >
```

- Subtask 2: Which synchronization mechanism(s) and method(s) are used to synchronize the Producer thread and the Consumer thread in this package?
 - Consumer and Producer are utilizing a queue buffer class which declare methods such as *put* and *remove* which are both synchronized. This provides intrinsic locks (monitor locks) for mutual exclusion between the Producer and Consumer threads.
- Subtask 3: Remove the synchronization of Producer and Consumer threads and explain how a data race occurs
 - Without synchronization, these threads access shared mutable state (storage) concurrently without any coordination, resulting in unpredictable behavior and potential program crashes.
 - LinkedList corruption: When Producer and Consumer threads call *add()* and *poll()* at the same time, they can corrupt the internal linked structure (pointers, head, tail) and potentially causing lost elements.
 - NullPointerException: When the queue is empty and Consumer calls *storage.poll().toString()*, *poll()* returns null, causing the exception.

Exercise 3: raceCondition

- Subtask 1: Explain why data race occurs
 - In general, Data races occur when multiple threads access shared memory concurrently without proper synchronization, and at least one access is a write operation. Since most operations are non-atomic at the CPU level (e.g., *count++* involves read-modify-write steps), threads can interleave their instructions unpredictably, leading to lost updates, inconsistent reads, and visibility issues due to CPU caching. Without synchronization mechanisms like locks or atomic operations, the program behaviour becomes non-deterministic—the outcome depends on thread scheduling timing, making the same code produce different results on different runs.
 - In this case, the code currently prevents data races by using a *ReentrantLock* in *DataRace.java*. The *ReentrantLock* ensures that only one thread can execute the entire read-modify-write sequence at a time, making the operations atomic and preventing the race condition.
- Subtask 2: Where are the critical sections
 - DataRace: The methods *increment* and *decrement* are critical sections in this class. Both critical sections are protected by the *ReentrantLock*, ensuring only one thread can execute either operation at a time. The *lock.lock()* and *lock.unlock()* calls define the boundaries of these critical sections.

- DecreaseTask: The call to `Counter.decrement()` in the loop is the critical section where the thread accesses the shared `DataRace` object. The synchronization is delegated to the `DataRace` class, where the `ReentrantLock` protects the actual modification of the count variable.
- IncreaseTask: The call to `counter.increment()` in the loop is the critical section where the thread accesses the shared `DataRace` object. The synchronization is delegated to the `DataRace` class, where the `ReentrantLock` protects the actual modification of the count variable.
- Subtask 3: Fix the data inconsistency problem using Monitor
 - Personally, I did not have issues with the provided code, but I still changed from `ReentrantLock` to `Monitor` with `synchronized` method, as seen below:



```

DataRace.java 1, M X
lab6 > src > raceCondition > DataRace.java > ...
You, now | 1 author (You)
1 package raceCondition;
2
3 You, now | 1 author (You)
4 public class DataRace {
5     private int count = 0;
6
7     public DataRace() {
8         count = 0;
9     }
10
11     public synchronized void increment() {
12         count++;
13     }
14
15     public synchronized void decrement() {
16         count--;
17     }
18     public int getCounter() {
19         return count;
20     }
21
22     @Override
23     public String toString() {
24         return "FinalValueCount [count=" + count + "]";
25     }
26 }
27

```

- Subtask 4: Fix the data inconsistency using Lock
 - o Again. The code I was provided was error-free and was implemented with lock-functionality from before, using *ReentrantLock*. Below is the code as I received it, using Lock.

```

DataRace.java 2, M X
lab6 > src > raceCondition > DataRace.java > ...
You, now | 1 author (You)
1 package raceCondition; DataRace.java is a non-project file,
2
3 import java.util.concurrent.locks.ReentrantLock;
4
5 You, now | 1 author (You)
6 public class DataRace {
7     private int count = 0;
8
9     public DataRace() {
10         count = 0;
11     }
12     private static ReentrantLock lock = new ReentrantLock();
13     public void increment() {
14         lock.lock();
15         try {
16             count++;
17         } finally {
18             lock.unlock();
19         }
20     }
21     public void decrement() {
22         lock.lock();
23         try {
24             count--;
25         } finally {
26             lock.unlock();
27         }
28     }
29     public int getCounter() {
30         return count;
31     }
32
33     @Override
34     public String toString() {
35         return "FinalValueCount [count=" + count + "]";
36     }
37 }
38

```

Exercise 4: parking

- Subtask 1: Run the two implementations – include outputs

- SafeMain:

```
Parallel-distributed-programming/lab6/parking on @ main [$X?]  
● > cd /home/emilb_linux/dev/V2026/Parallel-distributed-programming/lab6/parking  
javac -d bin src/safe/*.java src/main/SafeMain.java  
java -cp bin main.SafeMain  
Parking Simulator  
numberSensors=32  
Number of cars: 0  
Number of motorcycles: 0  
Closing accounting  
The total amount is : 640  
Parallel-distributed-programming/lab6/parking on @ main [$X?]  
○ > █
```

- UnsafeMain:

```
Parallel-distributed-programming/lab6/parking on @ main [$X?]  
● > cd /home/emilb_linux/dev/V2026/Parallel-distributed-programming/lab6/parking  
javac -d bin src/unsafe/*.java src/main/UnsafeMain.java  
java -cp bin main.UnsafeMain  
Parking Simulator  
  
numberSensors=32  
Number of cars: 0  
Number of motorcycles: 0  
Closing accounting  
The total ammount is : 634  
Parallel-distributed-programming/lab6/parking on @ main [$X?]  
○ > █
```

- Subtask 2: Expected output of *numberCars* and *numberMotorcycles*.

- SafeMain:

- Expected output: Both *numberCars* and *numberMotorcycles* should be 0
- The synchronization ensures thread-safe access, preventing race conditions that could occur in the unsafe version.

- UnsafeMain:

- Expected output: Unpredictable values due to race conditions (demonstrates why synchronization is needed)

- Subtask 3: Discuss the data race issue focusing on the use of the *synchronized* keyword
 - A data race occurs when multiple threads access shared variables without synchronization, leading to unpredictable behavior because operations like `numberCars++` are not atomic—they involve reading, modifying, and writing back the value. In the unsafe version, concurrent threads can interleave these steps, causing lost updates where one thread overwrites another's changes with stale data. The `synchronized` keyword solves this by ensuring mutual exclusion: only one thread can execute a synchronized block at a time by acquiring a lock (monitor) on the specified object. In `SafeParkingStats.java`, two separate locks (`controlCars` and `controlMotorcycles`) protect their respective variables, preventing data races while allowing concurrent modifications to different variables. This guarantees that all increments and decrements are properly counted, resulting in the correct output of 0 for both variables, whereas the unsafe version produces unpredictable, incorrect results due to race conditions.